Thin Keys, Full Values: Reducing KV Cache via Low-Dimensional Attention Selection

Anonymous authors
Paper under double-blind review

Abstract

Standard transformer attention uses identical dimensionality for queries, keys, and values, yet these components serve different roles: queries and keys produce scalar attention weights (*selection*), while values carry rich representations (*value transfer*). We show that selection requires only $\mathcal{O}(\log N)$ dimensions to distinguish among N relevant token categories (e.g., syntactic roles, semantic clusters, positional patterns)—far fewer than value transfer needs.

We introduce **factored keys**, which exploit this asymmetry to physically shrink the KV cache of *any pretrained model without retraining from scratch*—unlike GQA and MLA, which must be designed into the architecture before pretraining. We factorize each key projection $W_K \approx A_{d\times r}B_{r\times d}$ via truncated SVD, set $W'_K = A$ as the new key projection producing compact r-dimensional keys for the cache, and absorb $B^\top$ into the query projection ($W'_Q = W_Q B^\top$) at zero cost—since queries are never cached. At 7B scale, training from scratch with $d_{\text{select}} = d_{\text{model}}/4$ matches full-attention perplexity (9.2 vs 9.3 PPL after 20B tokens) while using 12% fewer parameters and training 8% faster. For existing models, SVD + QK fine-tuning (3 epochs, $<$1% of pretraining data) achieves 75% key cache savings at $\sim$2% quality cost on both GPT-2 and Mistral-7B. The approach composes with GQA and quantization for up to 16$\times$ combined key cache compression. For a 7B model serving 128K context, factored keys save 25 GB of KV cache per user, enabling $\sim$60% more concurrent users on identical hardware.

1 Introduction

The self-attention mechanism (Vaswani et al., 2017) projects inputs into queries, keys, and values (Q, K, V) and computes $\text{softmax}(QK^\top/\sqrt{d_k})\,V$. In virtually all modern transformers—GPT (Radford et al., 2018), LLaMA (Touvron et al., 2023), Mistral (Jiang et al., 2023)—the projection dimensionality is identical: $d_q = d_k = d_v = d_{\text{model}}$. This symmetry is a design convention, not a necessity.

As models scale to longer contexts, the KV cache becomes the dominant memory bottleneck during autoregressive inference: a 7B model serving 128K context requires over 60 GB of KV cache per user. Existing approaches to this bottleneck reduce the *number* of KV heads—Multi-Query Attention (Shazeer, 2019) and Grouped-Query Attention (Ainslie et al., 2023)—or compress the *joint* KV state into a low-rank latent. These methods have been widely adopted (GQA in LLaMA-2 (Touvron et al., 2023) onward; MLA in DeepSeek-V2 (Liu et al., 2024)), but they all maintain $d_k = d_{\text{head}}$ within each head's attention computation.

We take a complementary approach, motivated by a structural observation: attention performs two functionally distinct operations with fundamentally different complexity requirements. (1) **Selection** ($QK^\top$): determining *which* tokens are relevant via scalar attention weights. (2) **Value transfer** (attn $\cdot\, V$): aggregating information from selected tokens. Our key insight is that selection is a *ranking* problem, not a representation problem: by the Johnson–Lindenstrauss lemma (Johnson and Lindenstrauss, 1984), distinguishing among N items in a dot-product space requires only $\mathcal{O}(\log N)$ dimensions. Value transfer, by

contrast, must preserve the full representational capacity of the model. Recent theoretical work on KV cache compression lower bounds provides independent support: Haris and Onak (2025) show that any attention-based token generation algorithm requires $\Theta(nd)$ space with $d = \Omega(\log n)$, establishing that the KV cache cannot be compressed below $\log n$ dimensions per token without loss. This asymmetry implies that queries and keys are drastically overparameterized at $d_k = d_{\text{model}}$.

We propose **asymmetric attention**, where queries and keys project to $d_{\text{select}} \ll d_{\text{model}}$ while values retain full dimensionality. The attention computation is unchanged—$QK^\top / \sqrt{d_{\text{select}}}$ produces scalar weights applied to V of any dimensionality. This yields three benefits: (i) parameter reduction (up to 75% for QK when $d_{\text{select}} = d_{\text{model}}/4$), (ii) KV cache reduction (the key cache stores d_{select}-dimensional vectors, saving 25 GB per user at 128K context for 7B models), and (iii) attention compute reduction ($\mathcal{O}(n^2 \cdot d_{\text{select}})$ instead of $\mathcal{O}(n^2 \cdot d_{\text{model}})$). Unlike GQA or MLA, this approach reduces the *per-head dimensionality* of keys while keeping the number of heads and value dimensionality intact—and composes with both for further savings.

A central contribution of this work is that asymmetric attention can be applied to *any existing pretrained model* without retraining from scratch—in contrast to GQA and MLA, which must be built into the architecture before pretraining or require expensive conversion (Ma et al., 2025). We introduce **factored keys**, an inference primitive that physically shrinks the key cache of already-deployed models: factorize each layer's $W_K \approx A_{d\times r} B_{r\times d}$ via truncated SVD, use A as the new key projection producing compact r-dimensional cached keys, and absorb $B^\top$ into the query projection at zero memory cost—exploiting the fundamental asymmetry that keys accumulate in memory while queries are ephemeral. With optional lightweight QK fine-tuning (3 epochs on $<$1% of pretraining data), this recovers nearly all quality. This is related to recent work on low-rank KV cache compression (Li et al., 2025; Wang et al., 2024; Kang et al., 2024), but differs in a key respect: we compress *only* keys and absorb the cost into queries, which are never cached. A striking finding is that keys are far more compressible than queries: at rank 192 on GPT-2, K-only SVD degrades by 26% while Q-only degrades by 181%—a 7$\times$ asymmetry consistent with recent observations that key vectors lie in a significantly lower-dimensional space (Kang et al., 2024).

We validate across nine experiments from algorithmic tasks to 7B-scale training (Experiments 1–4 and 6 appear in Appendix A–B; the four largest-scale experiments are in the main text). Controlled experiments confirm that positional selection requires only 1 dimension per head, and content-based selection requires $\log_2 N$ dimensions (Appendix A). At 7B scale, training from scratch with $d_{\text{select}} = d_{\text{model}}/4$ matches full-attention perplexity after 20B tokens while using 12% fewer parameters and training 8% faster. For existing models, SVD + QK fine-tuning on Mistral-7B achieves 75% key cache savings at $\sim$2% quality cost.

2 Method

2.1 Asymmetric Attention

In multi-head attention with h heads, standard transformers set $d_{\text{head}} = d_{\text{model}}/h$ for all of Q, K, and V. We decouple this by introducing d_{select}, the total dimensionality for queries and keys:

$$d_{\text{head}}^{QK} = \frac{d_{\text{select}}}{h}, \qquad d_{\text{head}}^{V} = \frac{d_{\text{model}}}{h} \tag{1}$$

For each head i, the attention computation is:

$$Q_i = XW_Q^{(i)}, \quad W_Q^{(i)} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{head}}^{QK}} \tag{2}$$

$$K_i = XW_K^{(i)}, \quad W_K^{(i)} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{head}}^{QK}} \tag{3}$$

$$V_i = XW_V^{(i)}, \quad W_V^{(i)} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{head}}^{V}} \tag{4}$$

$$\text{head}_i = \text{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d_{\text{head}}^{QK}}}\right) V_i \tag{5}$$

No projection is needed between attention weights and value aggregation: the weights $\in \mathbb{R}^{n \times n}$ are scalars regardless of d_{head}^{QK}. When $d_{\text{select}} = d_{\text{model}}$, this reduces to standard multi-head attention.

2.2 Theoretical Motivation

We argue that selection requires $\mathcal{O}(\log N)$ dimensions where N is the number of distinct patterns the attention mechanism must distinguish. The attention weight $\alpha_{ij} \propto \exp(q_i^\top k_j / \sqrt{d})$ assigns scalar relevance scores—a ranking problem. By the Johnson–Lindenstrauss lemma, N points can be embedded into $\mathcal{O}(\log N)$ dimensions while preserving pairwise distances, so $\mathcal{O}(\log N)$ dimensions suffice to maintain the relative ordering of dot-product similarities. In language modeling, the effective N is the number of distinct selection patterns (syntactic roles, semantic clusters, recency patterns)—empirically in the hundreds, much smaller than vocabulary size, suggesting $d_{\text{select}} \approx 10$–$20$ dimensions per head. Value transfer, by contrast, must preserve the complete representational content across all d_{model} dimensions.

2.3 Factored Keys: An Inference Primitive for Pretrained Models

Given a pretrained $W_K \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$, we seek a thin-key factorization of the form:

$$\underbrace{\begin{bmatrix} W_K \end{bmatrix}}_{d \times d} \approx \underbrace{\begin{bmatrix} W_K' \end{bmatrix}}_{\textbf{thin keys } (d \times r)} \underbrace{\begin{bmatrix} B \end{bmatrix}}_{r \times d \text{ (absorbed into } W_Q)} \tag{6}$$

Conveniently, truncated SVD provides exactly this factorization, with $r = d_{\text{select}} \ll d_{\text{model}}$. We then replace the original projections:

$$W_K' = A = U_r \Sigma_r \in \mathbb{R}^{d_{\text{model}} \times r} \qquad (\textbf{thin key} \text{ projection — cached}) \tag{7}$$

$$W_Q' = W_Q B^\top = W_Q V_r \in \mathbb{R}^{d_{\text{model}} \times r} \qquad (\text{absorbed query projection — ephemeral}) \tag{8}$$

Keys become $k_j' = x_j W_K' \in \mathbb{R}^r$—the "thin keys" stored in the cache at a fraction of the original size. Queries become $q_i' = x_i W_Q' \in \mathbb{R}^r$, preserving attention scores exactly: $q_i' k_j'^\top = x_i W_Q B^\top A^\top x_j^\top = x_i W_Q W_K^\top x_j^\top = q_i k_j^\top$. Computing W_Q' is a one-time matrix multiplication—no training or fine-tuning is required. This is the beauty of SVD: we obtain thin keys from any pretrained full-key model *for free*, simply by repartitioning the existing weights. The result: key cache entries shrink from d_{model} to r dimensions while all compression cost is absorbed by the ephemeral query side (queries are computed fresh each step and never cached).

Crucially, *nothing else in the network changes*—W_V, W_O, the FFN, and all normalization layers remain untouched. The only modification is replacing W_K and W_Q with their thin counterparts, computed from a single SVD. Despite this simplicity, training from scratch

with thin keys at 7B scale achieves 9.2 vs 9.3 validation PPL after 20B tokens—matching full attention—while using 12% fewer parameters and training 8% faster (Experiments 7–7b, Tables 3 and 4).

This zero-cost property distinguishes factored keys from all existing KV compression methods. GQA (Ainslie et al., 2023) and MLA (Liu et al., 2024) must be designed into the architecture before pretraining—they cannot be applied to already-deployed models. LRQK (Li et al., 2025) performs QK decomposition at prefill time, adding latency to every inference call. SVD-LLM (Wang et al., 2024) requires calibration data and a whitening step to guide truncation. By contrast, factored keys require only a one-time offline SVD per layer—no calibration data, no prefill overhead, no retraining—and the resulting model runs with standard attention kernels. We validate this in Experiments 5 and 8: zero-cost SVD at rank $d_{\text{model}}/2$ immediately achieves 50% key cache savings at +2% PPL with no fine-tuning at all. To compress more aggressively, optional lightweight QK fine-tuning (3 epochs) recovers quality at lower ranks, enabling 75% savings at rank $d_{\text{model}}/4$ with only +2% PPL and 22–38% faster decode throughput on H100 (Tables 2, 6, and 10).

2.4 KV Cache Implications

During autoregressive generation, the KV cache for L layers at context length n is:

$$\text{Standard KV cache} = 2 \cdot n \cdot d_{\text{model}} \cdot L \cdot b \text{ bytes} \tag{9}$$

$$\text{Asymmetric KV cache} = n \cdot (d_{\text{select}} + d_{\text{model}}) \cdot L \cdot b \text{ bytes} \tag{10}$$

where b is bytes per parameter. With $d_{\text{select}} = d_{\text{model}}/4$, the K cache shrinks by 75%, yielding 37.5% total KV cache reduction.

3 Experiments

We validate asymmetric attention through nine experiments of increasing complexity. Controlled experiments on algorithmic tasks and small-scale language modeling (Experiments 1–4) are in Appendix A; architecture generalization at 125M scale (Experiment 6) is in Appendix B. The main text presents the four largest-scale experiments: post-training SVD compression of GPT-2 (Experiment 5), from-scratch training at 7B scale (Experiments 7 and 7b), and SVD + fine-tuning of Mistral-7B (Experiment 8).

3.1 Experiment 5: Post-Training Compression of GPT-2

Setup. We apply SVD compression to pretrained GPT-2 (124M parameters, $d_{\text{model}} = 768$, 12 heads, 12 layers) without retraining, testing three modes: both W_Q and W_K, K-only, and Q-only.

Results. Table 1 reveals a striking asymmetry. Compressing both W_Q and W_K is catastrophic—errors compound through the softmax. However, K-only compression is far more forgiving: rank 384 ($d_{\text{model}}/2$) incurs only 2.0% degradation. K is substantially more compressible than Q at every rank: at rank 192, K-only degrades by 26% while Q-only degrades by 181%—a 7× asymmetry.

Table 1: SVD compression of GPT-2 projections. Baseline PPL: 24.91. Δ is relative PPL increase.

Rank r	r/head	Both Q+K	K-only	Q-only
128	10	67,132	57.37 (+130%)	149.04 (+498%)
192	16	90.95 (+265%)	31.32 (+26%)	70.05 (+181%)
256	21	46.15 (+85%)	27.49 (+10%)	40.42 (+62%)
384	32	—	25.40 (+2.0%)	27.58 (+11%)
512	42	25.69 (+3.1%)	25.23 (+1.3%)	25.47 (+2.2%)

Deployment via factored keys. The SVD factorization provides a direct path to KV cache reduction: the key cache stores r-dimensional vectors while $B^\top$ is absorbed into the query projection. Our K-only PPL measurements directly reflect deployment performance: rank 384 ($d_{\text{model}}/2$) saves 50% of the key cache (25% total KV) at +2.0% PPL; rank 512 saves 33% of keys at +1.3%.

Recovery via QK fine-tuning. We compress W_K via SVD, then fine-tune only QK projections (∼21M of 124M parameters) on WikiText-103 for 3 epochs over 10M tokens. Table 2 shows that fine-tuning nearly eliminates the SVD quality loss: at rank 192 ($d_{\text{model}}/4$), the gap shrinks from +27.6% to just +1.8% relative to an identically fine-tuned control.

Table 2: SVD compression + QK fine-tuning on WikiText-103. The "vs control" column measures the residual gap relative to the uncompressed model after both receive identical fine-tuning.

Rank r	Before FT	After FT	Control	vs Control	K cache saved
768 (none)	29.51	19.07	—	baseline	0%
384 ($d_{\text{model}}/2$)	30.15 (+2.2%)	19.14	19.07	+0.4%	50%
256 ($d_{\text{model}}/3$)	32.61 (+10.5%)	19.28	19.07	+1.1%	67%
192 ($d_{\text{model}}/4$)	37.64 (+27.6%)	19.42	19.07	+1.8%	75%
128 ($d_{\text{model}}/6$)	67.26 (+128%)	19.77	19.07	+3.7%	83%

3.2 Experiment 7: From-Scratch Training at 7B Scale

Setup. We train two LLaMA-7B models from random initialization on OpenWebText (2B tokens): a full-attention baseline and a thin-keys variant with $d_{\text{select}} = 1024$ ($d_{\text{model}}/4$). Both use $d_{\text{model}} = 4096$, 32 heads, 32 layers, $d_{\text{ff}} = 11008$, with identical hyperparameters. We run each configuration with two random seeds and report mean ± std.

Table 3: 7B LLaMA trained from scratch on OpenWebText (2B tokens), mean ± std over 2 seeds.

	Full Attention	Thin Keys ($d_{\text{select}} = d_{\text{model}}/4$)
Parameters	6.74B	5.93B (−12%)
OWT val PPL	13.82 ± 0.09	$\mathbf{13.12 \pm 0.04}$ (−5.1%)
WT103 val PPL	20.79 ± 0.35	$\mathbf{19.60 \pm 0.46}$ (−5.7%)
Wall-clock time	25.9h	23.4h (−9.4%)

Results. Thin keys *outperform* full attention at 7B scale with 2B tokens (Table 3): 5.1% better OWT perplexity while using 12% fewer parameters and training 9.4% faster. This inverts the ∼4% cost observed at smaller scales (Appendix A, B), consistent with a regularization effect in the overparameterized regime (tokens-to-parameters ratio ∼0.3). The training trajectory (Figure 1) shows thin keys leading at every checkpoint.

Step	Full	Thin
5K	26.62	25.05
10K	20.81	19.21
15K	17.68	16.51
20K	15.66	14.74
25K	14.37	13.59
30K	13.79	13.12

Step	Full	Thin
5K	44.23	41.34
10K	31.99	29.20
15K	26.43	25.36
20K	24.20	22.73
25K	22.05	20.83
30K	21.19	19.96

Figure 1: Training trajectory for 7B from-scratch models (seed 137). Left: OWT validation PPL. Right: WikiText-103 validation PPL. Thin keys lead throughout.

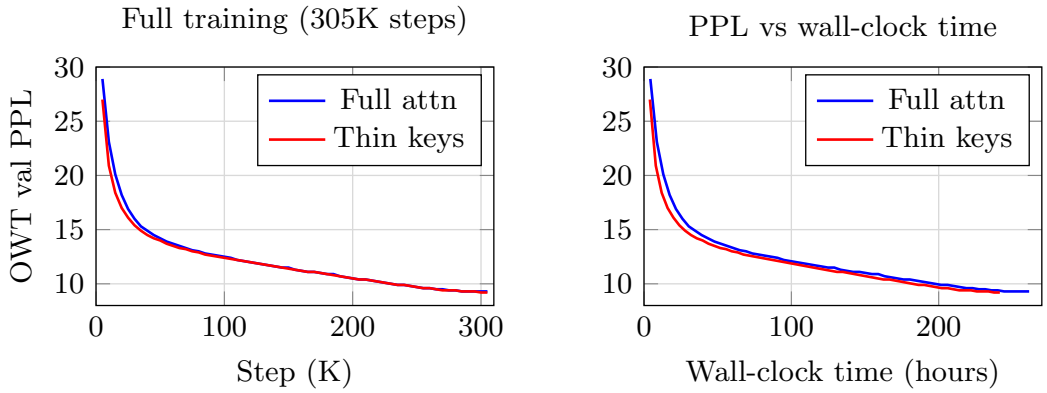

Figure 2: Training trajectory for 7B models over 305K steps (20B tokens). **Left:** PPL vs step; thin keys lead early but converge by ∼150K steps. **Right:** PPL vs wall-clock time; thin keys reach any target PPL ∼20 hours sooner.

3.3 Experiment 7b: Extended Training at 7B Scale (20B Tokens)

Setup. We extend training to 20B tokens (3 epochs over OpenWebText, 305,175 steps), increasing the tokens-to-parameters ratio to ∼3.0. Architecture and hyperparameters are identical to Experiment 7.

Table 4: 7B LLaMA on OpenWebText (20B tokens). Both architectures converge to the same quality; thin keys train 8% faster.

	Full Attention	Thin Keys ($d_{\text{select}} = d_{\text{model}}/4$)
Parameters	6.74B	5.93B (−12%)
OWT val PPL	9.3	**9.2** (−1.1%)
Wall-clock time	261.6h	241.6h (−7.7%)

Results. At 20B tokens, both models converge to nearly identical perplexity (9.2 vs 9.3), confirming the earlier advantage was a regularization effect. Crucially, thin keys *never fall behind*. Figure 2 shows PPL vs training step (curves merge by ∼150K steps) and vs wall-clock time (thin keys reach any target PPL ∼20 hours sooner). Combined with 8% speedup and 75% key cache savings, thin keys are a strictly dominant choice at this scale.

Comparison with GQA and MLA at 7B scale. Table 5 compares thin keys against GQA and MLA analytically at the LLaMA-7B configuration ($d_{\text{model}} = 4096$, 128K context, bf16). GQA and MLA compress both K and V, achieving larger total savings (75–93%) than thin keys alone (37.5%). However, thin keys composes with GQA: applying $d_{\text{select}} = d_{\text{model}}/4$ to GQA-8 yields 84.4% total KV savings—approaching MLA's 93% without learned up/down-projections or decoupled RoPE. MLA's joint latent means thin-keys composition is a no-op, but the thin-keys insight is *already embedded* in MLA: DeepSeek-V2's $d_c = 512$ for 128 heads implies an effective per-head key dimension of 4, far below $d_{\text{head}} = 128$. For practitioners, factored keys offer the simplest drop-in path for existing models; for new architectures, GQA + thin keys or MLA achieve more aggressive compression (Ma et al., 2025).

3.4 Experiment 8: SVD + Fine-tuning at Scale (Mistral 7B)

Setup. We apply SVD + QK fine-tuning to Mistral-7B (7.2B parameters, GQA with 32 query heads, 8 KV heads, $d_{\text{model}} = 4096$). We compress W_K via SVD to ranks 512, 256, and 128, then fine-tune only QK projections on WikiText-103 for 3 epochs (10M tokens).

Table 5: Analytical KV cache comparison at LLaMA-7B scale (128K context, bf16). Thin keys compose with GQA by reducing d_k independently.

Method	K cache (GB)	V cache (GB)	KV total (GB)	KV saved
MHA (baseline)	32.0	32.0	64.0	—
Thin keys ($d_{\text{select}} = d_{\text{model}}/4$)	8.0	32.0	40.0	37.5%
GQA-8	8.0	8.0	16.0	75.0%
MLA ($d_c = 512$, $d_h^R = 64$)	4.5 (joint)		4.5	93.0%
GQA-8 + thin keys	2.0	8.0	10.0	84.4%

Table 6: Mistral-7B: SVD compression + QK fine-tuning on WikiText-103.

Rank r	Before FT	After FT	Control	vs Control	K cache saved
1024 (none)	6.69	5.91	—	baseline	0%
512 ($k/2$)	6.81 (+1.7%)	5.93	5.91	+0.3%	50%
256 ($k/4$)	7.84 (+17.1%)	6.03	5.91	+2.0%	75%
128 ($k/8$)	12.34 (+84.4%)	6.10	5.91	+3.2%	88%

Results. The pipeline scales to 7B with consistent quality (Table 6). At rank 256 (75% K cache saved), the residual gap is +2.0%—matching GPT-2 results exactly.

Downstream task evaluation. We evaluate on five benchmarks: Hellaswag (10-shot), ARC-Challenge (25-shot), WinoGrande (5-shot), MMLU (5-shot), and GSM8K (5-shot CoT). Table 7 shows the results.

Table 7: Downstream evaluation of SVD-compressed Mistral-7B. "Ctrl+FT" is the uncompressed model with identical fine-tuning.

Task	Metric	Baseline	r512 +FT	r256 +FT	Ctrl+FT	Δ_{512}	Δ_{256}
Hellaswag	acc_norm	81.2	81.4	80.7	81.3	+0.1	−0.7
ARC-Challenge	acc_norm	54.0	54.1	53.4	54.4	−0.5	−1.7
WinoGrande	acc	75.4	73.2	72.1	73.2	+0.0	−1.6
MMLU	acc	60.1	55.2	54.4	55.7	−0.9	−2.3
GSM8K	exact_match	38.4	27.7	25.8	29.9	−7.4	−13.7

At rank 512, compression is effectively lossless on knowledge and commonsense tasks (<1% vs control). At rank 256, gaps remain modest (0.7–2.3%) except for GSM8K, where multi-step math reasoning is more sensitive. Domain-matched fine-tuning recovers GSM8K quality: fine-tuning on GSM8K's own training split closes the compression gap from −13.7% to just −1.2% for r=256 (Appendix D). Generalization to held-out math benchmarks (Minerva Algebra, AGIEVAL AQuA-RAT) confirms transferable reasoning recovery with <2.5% compression gaps.

4 Analysis

4.1 Practical Benefit: KV Cache

The primary practical benefit is inference-time KV cache reduction. For a representative 7B configuration ($d_{\text{model}} = 4096$, 32 layers, fp16), Table 9 shows savings at different context lengths.

These savings compound with concurrent users and context length. Even the conservative SVD path saves 131 GB per user at 1M context.

Table 8: Math generalization of GSM8K-fine-tuned models on held-out benchmarks.

Benchmark	Metric	Control	r512	r256	Δ_{512}	Δ_{256}
GSM8K	exact_match	53.7	52.1	51.2	-1.6	-2.5
Minerva Algebra	math_verify	14.2	14.1	12.4	-0.2	-1.9
Minerva Pre-Algebra	math_verify	21.0	19.1	19.1	-2.0	-2.0
AGIEVAL AQuA-RAT	acc	15.7	17.3	17.3	$+1.6$	$+1.6$

Table 9: KV cache memory per user ($d_{\text{model}} = 4096$, 32 layers, fp16).

	Standard	$d_{\text{select}} = d_{\text{model}}/2$ (SVD, no retrain)	$d_{\text{select}} = d_{\text{model}}/4$ (train or SVD+FT)
128K context			
K cache	33.6 GB	16.8 GB	8.4 GB
V cache	33.6 GB	33.6 GB	33.6 GB
Total	**67.2 GB**	**50.4 GB**	**42.0 GB**
Savings/user	—	16.8 GB (25%)	25.2 GB (37.5%)
1M context			
Total	**524 GB**	**393 GB**	**328 GB**
Savings/user	—	131 GB (25%)	196 GB (37.5%)

4.2 Decode Throughput: Roofline Analysis

The KV cache savings translate directly to decode throughput gains. Table 10 reports measured decode throughput for Mistral-7B on a single H100 SXM GPU, showing 22–38% speedups at batch size ≥ 4.

Table 10: Measured decode throughput (tokens/s) for Mistral-7B with factored keys on H100 SXM (context 4096, 128 generated tokens).

	Batch size				
Config	1	4	8	16	32
Throughput (tokens/s)					
Baseline ($d_k = 128$)	45.4	117.4	149.2	171.9	187.3
r512 ($d_k = 64$)	54.5	145.1	181.9	210.9	230.2
r256 ($d_k = 32$)	46.0	156.5	200.5	234.9	259.3
Speedup vs. baseline					
r512	1.20×	1.24×	1.22×	1.23×	1.23×
r256	1.01×	1.33×	1.34×	1.37×	1.38×

Decode attention is deeply memory-bandwidth-bound: on H100 SXM (3.35 TB/s bandwidth), decode attention operates 74× below the compute ridge point. Throughput scales directly with bytes saved. Amdahl's law (attention fraction $f \approx 0.73$ at batch ≥ 8) predicts 1.38× overall speedup for r256—matching measured 1.38× at batch 32 to within 1%. Standard SDPA suffices; no custom Flash Attention kernels are required. Prefill roofline analysis is in Appendix E.

5 Related Work

Multi-Query, Grouped-Query, and Multi-Latent Attention. MQA (Shazeer, 2019) and GQA (Ainslie et al., 2023) reduce KV cache by sharing KV heads across query groups. MLA (Liu et al., 2024) projects the joint KV state into a low-dimensional latent. These methods reduce head count or joint KV dimensionality; our approach reduces *per-head key*

dimensionality and composes with all three (Table 16 in Appendix B). Chen et al. (2025) show commonly used GQA configurations are suboptimal for long contexts.

Low-Rank Attention and KV Cache Compression. Linformer (Wang et al., 2020) reduces the sequence dimension; we reduce the feature dimension. Loki (Kang et al., 2024) exploits low-rank key structure for sparse attention. LRQK (Li et al., 2025) jointly decomposes QK matrices during prefill. SVD-LLM (Wang et al., 2024) applies truncation-aware SVD for post-training compression. KVQuant (Hooper et al., 2024) and AsymKV (Tao et al., 2024) reduce bit width; ZACK (Zhang et al., 2025) achieves zero-overhead dimensionality compression. Haris and Onak (2025) prove $\Omega(\log n)$ space lower bounds for attention. Our dimensionality reduction is orthogonal to quantization and composes multiplicatively.

Efficient Attention. Flash Attention (Dao et al., 2022; Shah et al., 2024) optimizes memory access patterns; sparse attention methods (Child et al., 2019; Beltagy et al., 2020; Mazaré et al., 2025) reduce the attended set. These are complementary to reducing QK dimensionality.

6 Discussion

Limitations and scope. We validate training from scratch up to 7B parameters over 20B tokens. At this budget (tokens-to-parameters $\sim$3), thin keys match full attention; at truly Chinchilla-optimal budgets, a modest cost may emerge as at smaller scales. While we evaluate perplexity and five downstream tasks, other capabilities (in-context learning, instruction following, long-context retrieval) may exhibit different sensitivity.

Flash Attention and composability. Most Flash Attention implementations assume $d_k = d_v$, but standard SDPA already achieves 22–38% decode speedups (Table 10)—decode is bandwidth-bound, so standard kernels suffice. Thin keys compose with KV quantization (Liu et al., 2024; Hooper et al., 2024; Tao et al., 2024): dimensionality reduction removes low-rank redundancy, then quantization compresses remaining elements, yielding up to 16$\times$ combined key cache compression (4$\times$ from thin keys $\times$ 4$\times$ from INT4).

Implications for architecture design. The standard convention $d_q = d_k = d_v$ should be reconsidered. Setting $d_{\text{select}} = d_{\text{model}}/4$ yields consistent savings across architectures and scales. We identify three deployment paths: (1) K-only SVD at $d_{\text{model}}/2$ for 25% savings with zero retraining; (2) SVD + QK fine-tuning to $d_{\text{model}}/4$ for 75% savings at $\sim$2% cost; (3) training from scratch with thin keys for future architectures—analogous to how GQA was adopted into LLaMA-2 onward.

7 Conclusion

We identify a fundamental asymmetry in attention: selection ($QK^\top$) is an inherently low-dimensional ranking problem requiring only $\mathcal{O}(\log N)$ dimensions, while value transfer must preserve the full representational capacity of the model. From this insight, we derive **factored keys**, a zero-cost inference primitive that exploits SVD to physically shrink the key cache of any deployed model—no retraining, no calibration data, no prefill overhead—by repartitioning W_K into a thin cached key projection and absorbing the remainder into the ephemeral query projection.

We validate this across nine experiments from algorithmic tasks to 7B-parameter models trained on 20B tokens. At 7B scale, training from scratch with $d_{\text{select}} = d_{\text{model}}/4$ matches full-attention perplexity (9.2 vs 9.3) while using 12% fewer parameters and training 8% faster. For existing models, SVD + lightweight QK fine-tuning achieves 75% key cache savings at $\sim$2% cost across a 58$\times$ scale range (GPT-2 to Mistral-7B). The approach composes with GQA and quantization for up to 16$\times$ combined compression.

References

Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., and Sanghai, S. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *EMNLP*, 2023.

Beltagy, I., Peters, M. E., and Cohan, A. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

Chen, Y., Wu, Y., Song, C., Thai, Z. L., Shen, X., Han, X., Liu, Z., and Sun, M. Cost-optimal grouped-query attention for long-context modeling. In *EMNLP*, 2025.

Child, R., Gray, S., Radford, A., and Sutskever, I. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *NeurIPS*, 2022.

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*, 2019.

Frankle, J. and Carlin, M. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *ICLR*, 2019.

Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M. W., Shao, Y. S., Keutzer, K., and Gholami, A. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *NeurIPS*, 2024.

Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M. W., Shao, Y. S., Keutzer, K., and Gholami, A. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *NeurIPS*, 2024.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.

Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., et al. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023.

Johnson, W. B. and Lindenstrauss, J. Extensions of Lipschitz mappings into a Hilbert space. *Contemporary Mathematics*, 26:189–206, 1984.

Kang, J., et al. Loki: Low-rank keys for efficient sparse attention. *arXiv preprint arXiv:2406.02542*, 2024.

Kitaev, N., Kaiser, Ł., and Levskaya, A. Reformer: The efficient transformer. In *ICLR*, 2020.

Li, T., Zhou, G., Zhao, X., Qiu, Y., and Zhao, Q. Efficient low rank attention for long-context inference in large language models. In *NeurIPS*, 2025.

Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., and Hu, X. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *ICML*, 2024.

Liu, A., Feng, B., Wang, B., et al. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

Ma, J., Zhou, H., Lin, J., Wang, Z., Hu, J., and Han, S. Towards economical inference: Enabling DeepSeek's MLA in any transformer. *arXiv preprint arXiv:2502.14837*, 2025.

Mazaré, P.-E., Szilvasy, G., Lomeli, M., Massa, F., Murray, N., Jégou, H., and Douze, M. Inference-time sparse attention with asymmetric indexing. *arXiv preprint arXiv:2502.08246*, 2025.

Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models. In *ICLR*, 2017.

Radford, A., Narasimhan, K., Salimans, T., and Sutskever, I. Improving language understanding by generative pre-training. 2018.

Shah, J., et al. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. *arXiv preprint arXiv:2407.08608*, 2024.

Shazeer, N. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

Tao, Q., et al. AsymKV: Enabling 1-bit quantization of KV cache with layer-wise asymmetric quantization configurations. *arXiv preprint arXiv:2410.13212*, 2024.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. In *NeurIPS*, 2017.

Wang, S., Li, B. Z., Khabsa, M., Fang, H., and Ma, H. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

Wang, X., et al. SVD-LLM: Truncation-aware singular value decomposition for large language model compression. In *ICLR*, 2025.

Zhang, Y., Hu, Y., Zhao, R., Lui, J. C. S., and Chen, H. Unifying KV cache compression for large language models with LeanKV. *arXiv preprint arXiv:2412.03131*, 2024.

Zhang, Z., et al. ZACK: Zero-overhead LLM inference acceleration via dimensionality compression of the key-value cache. *arXiv preprint arXiv:2408.04107*, 2025.

Haris, A. and Onak, K. Compression barriers for autoregressive transformers. *arXiv preprint arXiv:2502.15955*, 2025.

A Small-Scale Experiments

Experiments 1–4 use a standard transformer decoder with pre-norm layer normalization, GELU activations, and learned positional embeddings. The only variable across configurations is d_{select}.

A.1 Experiment 1: Positional Selection (Copy-Back Task)

Setup. We design a task where each token must copy the token from $K = 8$ positions earlier: $y_t = x_{t-K}$. The source position is fixed regardless of token content, isolating purely *positional* selection. Sequences of length 64 are generated with random tokens from a vocabulary of 16. The model uses $d_{\text{model}} = 64$, 4 heads, 2 layers.

Results. Table 11 shows that even $d_{\text{select}} = 4$ (1 dimension per head) achieves 100% accuracy, though convergence is slightly slower. This confirms that positional selection—learning to attend to a fixed offset—requires minimal dimensionality.

A.2 Experiment 2: Content-Based Selection (Key-Value Retrieval)

Setup. We construct sequences of 8 random key-value pairs from a vocabulary of 16 tokens, followed by a query key. The model must output the value associated with the queried key. Positions are randomized each batch, so positional information is useless; the model must match content. Architecture: $d_{\text{model}} = 64$, 4 heads, 4 layers.

Table 11: Copy-back task: accuracy and convergence by d_{select}.

d_{select}	d_{select}/head	Best Accuracy	Converge Epoch
4	1	100.0%	300
8	2	100.0%	200
16	4	100.0%	200
32	8	100.0%	200
64	16	100.0%	200

Results. Table 12 reveals a sharp transition between $d_{\text{select}} = 4$ and $d_{\text{select}} = 8$. With 1 dimension per head, keys map to scalars and cannot be reliably separated via dot product, achieving only 65.2% accuracy. With 2 dimensions per head, keys can point in distinct angular directions, enabling perfect matching. The $\mathcal{O}(\log_2 N)$ prediction gives a lower bound: 16 keys require $\log_2(16) = 4$ dimensions total, but the model needs a small constant factor above this minimum to learn reliable separation via gradient descent. In practice, $d_{\text{select}} \approx 2 \log_2 N$ appears sufficient.

Table 12: Key-value retrieval: accuracy and convergence by d_{select}.

d_{select}	d_{select}/head	Best Accuracy	Converge Epoch
4	1	65.2%	did not converge
8	2	100.0%	1900
16	4	100.0%	1900
32	8	100.0%	1400
64	16	100.0%	1000

A.3 Experiment 3: WikiText-2 Language Modeling

Setup. We train causal language models on WikiText-2 (Merity et al., 2017) (2M training tokens) with $d_{\text{model}} = 256$, 8 heads, 6 layers, $d_{\text{ff}} = 1024$, sweeping $d_{\text{select}} \in \{8, 16, 32, 64, 128, 256\}$. Word-level tokenization with vocabulary size ∼29K (min_freq = 2). Training uses AdamW with cosine learning rate schedule for 30 epochs.

Results. Table 13 shows that $d_{\text{select}} = 64$ ($d_{\text{model}}/4$) achieves a test perplexity of 122.24, essentially identical to the baseline of 122.22. Even $d_{\text{select}} = 32$ loses only 1.3%. Notably, $d_{\text{select}} = 128$ *outperforms* the baseline (120.76 vs 122.22), likely because reducing QK capacity acts as a regularizer in this heavily overfitting regime (train PPL 37 vs validation PPL 127).

Connecting to the $\mathcal{O}(\log N)$ prediction. The vocabulary contains ∼29K words, yet $d_{\text{select}} = 64$ (8 dimensions per head) suffices. This implies that the effective number of selection patterns N is far smaller than the vocabulary size. Intuitively, attention does not need to distinguish every individual word from every other—it needs to distinguish *categories* of tokens relevant to different attention patterns: syntactic roles (subject vs. object vs. modifier), semantic clusters (topic-related words), positional patterns (recent vs. distant tokens), and punctuation or structural cues. The number of such categories is in the hundreds, not the tens of thousands. With 8 heads each operating in 8 dimensions, the model can represent $\sim 2^8 = 256$ distinguishable patterns per head, which appears to be sufficient for the selection demands of language modeling. This is consistent with the observation that attention heads tend to specialize into a modest number of interpretable roles (positional, syntactic, rare-word) rather than implementing vocabulary-sized lookup tables.

A.4 Experiment 4: WikiText-103 Language Modeling

Setup. To eliminate overfitting as a confound, we repeat the experiment on WikiText-103 (Merity et al., 2017) (100M training tokens) with the same architecture. Word-level

Table 13: WikiText-2 results. Model: $d_{\text{model}} = 256$, 8 heads, 6 layers. Baseline $d_{\text{select}} = 256$.

d_{select}	d_{select}/head	Val PPL	Test PPL	ΔPPL	QK Params	QK Saved
8	1	133.78	126.48	+3.5%	24,672	97%
16	2	132.67	125.49	+2.7%	49,344	94%
32	4	130.51	123.78	+1.3%	98,688	87%
64	8	129.34	122.24	+0.0%	197,376	75%
128	16	126.42	120.76	−1.2%	394,752	50%
256	32	126.95	122.22	—	789,504	0%

tokenization with min_freq = 200 yields a vocabulary of ∼22K. We train for 10 epochs with batch size 64 and 2000 warmup steps.

Results. Table 14 shows the clean comparison. With 50× more training data, the model is capacity-limited (train PPL ≈ val PPL), eliminating the regularization confound. Here $d_{\text{select}} = 128$ ($d_{\text{model}}/2$) incurs only a 2.1% perplexity increase with 50% QK savings, and $d_{\text{select}} = 64$ ($d_{\text{model}}/4$) incurs 4.3% for 75% savings—larger than on WikiText-2, confirming that the WikiText-2 result was partly masked by overfitting. Nevertheless, the tradeoffs remain compelling: the relationship between d_{select} and perplexity is smooth and monotonic, allowing practitioners to choose their operating point on a clear Pareto frontier.

Table 14: WikiText-103 results. Model: $d_{\text{model}} = 256$, 8 heads, 6 layers. Baseline $d_{\text{select}} = 256$.

d_{select}	d_{select}/head	Val PPL	Test PPL	ΔPPL	QK Saved
32	4	38.38	—	+7.6%	87%
64	8	37.22	—	+4.3%	75%
128	16	36.42	35.80	+2.1%	50%
256	32	35.67	—	—	0%

B Architecture Generalization at 125M Scale

B.1 Experiment 6: Architecture Generalization (LLaMA 125M)

Motivation. All training experiments so far use 10M-parameter vanilla transformers. We validate on a faithful LLaMA implementation at 125M parameters—12.5× larger.

Setup. We implement a standard LLaMA architecture with RMSNorm, SwiGLU FFN, Rotary Position Embeddings (RoPE), no bias terms, pre-norm residuals, and tied embeddings (Touvron et al., 2023). The *only* modification: W_Q and W_K project to d_{select} dimensions; W_V remains d_{model}. When $d_{\text{select}} = d_{\text{model}}$, the model is exactly standard LLaMA. Configuration: $d_{\text{model}} = 768$, 12 heads, 12 layers, $d_{\text{ff}} = 2048$, ∼101.7M parameters at baseline. Training: WikiText-103 with vocabulary truncated to 22K tokens (min frequency 200), 5 epochs, batch size 64, sequence length 512, cosine schedule with 2000-step warmup.

Table 15: LLaMA 125M with asymmetric attention on WikiText-103. Comparison with 10M vanilla transformer from Experiment 4.

d_{select}	d_{select}/head	Params	Val PPL	ΔPPL	QK saved
768 (full)	64	101.7M	22.80	—	0%
192 ($d_{\text{model}}/4$)	16	91.1M	23.77	+4.3%	75%
96 ($d_{\text{model}}/8$)	8	89.3M	24.45	+7.2%	87%
48 ($d_{\text{model}}/16$)	4	88.4M	25.30	+11.0%	94%

Results. The degradation ratios are remarkably consistent across architectures and scales:

d_{select}	10M Vanilla (Exp. 4)	125M LLaMA (Exp. 6)
$d_{\text{model}}/4$	+4.3%	+4.3%
$d_{\text{model}}/8$	+7.6%	+7.2%

The +4.3% cost at $d_{\text{select}} = d_{\text{model}}/4$ is identical across architectures despite a 12.5× scale difference and completely different design choices (LayerNorm vs RMSNorm, GELU vs SwiGLU, learned positions vs RoPE). This strongly suggests the QK dimensionality requirement is a fundamental property of the attention mechanism.

Comparison with alternative KV compression methods. We train the same 125M LLaMA architecture with Grouped-Query Attention (GQA) (Ainslie et al., 2023) and Multi-Latent Attention (MLA) (Liu et al., 2024). All models are trained from scratch with identical hyperparameters.

Table 16: 125M LLaMA: comparison of KV compression methods trained from scratch on WikiText-103. KV budget = per-token, per-layer cache size (K + V dimensions stored).

Method	Config	Params	KV budget	KV saved	Test PPL
MHA	12 heads	101.7M	1536	0%	23.07
Thin keys	$d_{\text{select}} = 384$	94.6M	1152	25%	23.22 (+0.7%)
Thin keys	$d_{\text{select}} = 192$	91.1M	960	37.5%	24.09 (+4.4%)
GQA	6 KV heads	94.6M	768	50%	23.15 (+0.3%)
GQA	4 KV heads	92.3M	512	66.7%	23.32 (+1.1%)
MLA	$d_c = 768$	108.8M	768	50%	23.11 (+0.2%)
MLA	$d_c = 512$	101.7M	512	66.7%	23.23 (+0.7%)

Table 16 shows that all three approaches achieve strong compression with modest quality costs. Thin keys with $d_{\text{select}} = 384$ achieves 23.22 test PPL—matching GQA-4 and MLA-512 quality despite compressing only keys. The practical implication is that thin keys *compose* with GQA or MLA for further savings.

C Additional Analysis

C.1 Scaling of Minimum d_{select}

Our experiments reveal a consistent pattern in the minimum d_{select} required for different selection tasks:

Table 17: Minimum effective d_{select} scales with task complexity.

Task	$N_{\text{effective}}$	Min d_{select}/head	Prediction ($\log_2 N$)
Positional (copy-back)	∼10 offsets	1	$\log_2 10 \approx 3$
Content (16 keys)	16 keys	2	$\log_2 16 = 4$ (total)
Language (WikiText)	∼256 patterns	8	$\log_2 256 = 8$

The empirical results are consistent with the $\mathcal{O}(\log N)$ prediction. For language modeling, the effective N appears to be approximately 256, suggesting that attention selection operates over a few hundred distinct semantic/syntactic categories rather than the full vocabulary space. This aligns with recent findings that key vectors naturally lie in a significantly lower-dimensional space than the model dimension (Kang et al., 2024).

C.2 Overfitting Masks the Effect

The WikiText-2 vs WikiText-103 comparison reveals an important methodological point: on small datasets, reducing d_{select} can *appear* costless (or even beneficial) because the model is overfitting. The WikiText-2 baseline has a train/val PPL ratio of 3.4×, indicating massive overfitting. Removing QK capacity acts as implicit regularization. On WikiText-103, where the model is underfitting (train PPL $>$ val PPL), the true cost of reduced d_{select} becomes visible.

This suggests that results reported only on small benchmarks may overstate the losslessness of QK compression, and that large-scale experiments are essential.

D GSM8K Fine-tuning Progression

Table 18 shows the full progression of GSM8K recovery across fine-tuning experiments. Fine-tuning on out-of-domain data (WikiText-103, C4) yields GSM8K scores in the 22–32% range with large compression gaps. Domain-matched fine-tuning on GSM8K's own training split (Experiment F3, 7,473 chain-of-thought examples, ∼1.5M tokens) more than doubles scores and closes the compression gap for r=256 from -13.7% to just -1.2%.

Table 18: GSM8K exact-match accuracy across fine-tuning experiments. All models use the same QK-only fine-tuning protocol (3 epochs, lr=5×10^{-5}). "Control" = uncompressed model with identical fine-tuning.

Exp	FT Data	Control	r512	r256	Δ_{512}	Δ_{256}
–	None (baseline)	38.4	33.8	16.5	–	–
A	WikiText-103	29.9	27.7	25.8	−7.4	−13.7
F	C4 (10M tok)	30.6	29.4	22.8	−3.9	−25.5
F2	C4 + Math (10M tok)	31.7	28.2	24.1	−3.5	−7.6
F3	GSM8K CoT (1.5M tok)	**53.7**	**52.5**	**52.0**	**−0.7**	**−1.2**

This demonstrates that the GSM8K degradation was a fine-tuning data problem, not a fundamental compression limitation. Data quality and domain match matter far more than data volume: 1.5M tokens of in-domain CoT outperform 10M tokens of generic web text.

E Prefill Roofline and Flash Attention

Prefill roofline. Prefill differs from decode: the full $QK^\top$ product is computed over the entire prompt. At context length $s = 4096$ with Mistral-7B, a single layer's attention FLOPs are ∼137 GFLOPs while KV reads are ∼2 MB—yielding arithmetic intensity of ∼68,000 FLOP/byte, well above the H100's ridge point. Prefill attention is thus compute-bound. Reducing d_k from 128 to 32 cuts $QK^\top$ FLOPs by 4× per head. Standard FlashAttention-2 assumes $d_k = d_v$ for shared tile sizes; FlashAttention-3 (Shah et al., 2024) introduces more flexible tiling that could accommodate asymmetric dimensions. Standard SDPA with `enable_math=True` already achieves 6–12% prefill speedups.